Java Database Connectivity (JDBC) Lab Report

1. Database Setup & Driver Prerequisites

To run these JDBC programs, ensure you have the MySQL Connector/J driver added to your project's classpath and a local database instance running. Below is the configuration used for these tasks:

```
Database URL: jdbc:mysql://localhost:3306/lab_db
Username: root
Password: password123
```

2. Task 1: Insert and Display Data using PreparedStatement

Objective: Create a table named 'Person' (containing Department ID, Department Name, and City) and write a program to insert rows and display all table contents using a PreparedStatement.

SQL Table Creation Query:

```
CREATE TABLE Person (
    Department INT,
    Department_Name VARCHAR(255),
    City VARCHAR(255)
);
```

Java Source Code (InsertAndDisplay.java):

```
import java.sql.*;
import java.util.Scanner;

public class InsertAndDisplay {
    private static final String URL = "jdbc:mysql://localhost:3306/lab_db";
    private static final String USER = "root";
    private static final String PWD = "password123";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, USER, PWD)) {
            System.out.println("Connected to database successfully.");

            // 1. Insert Data using PreparedStatement
            String insertSQL = "INSERT INTO Person (Department, Department_Name, City)
VALUES (?, ?, ?)";
            try (PreparedStatement pstmt = conn.prepareStatement(insertSQL);
                 Scanner scanner = new Scanner(System.in)) {
```

```
                System.out.print("Enter Department ID (int): ");
                int deptId = scanner.nextInt();
                scanner.nextLine(); // consume newline
                System.out.print("Enter Department Name: ");
                String deptName = scanner.nextLine();
                System.out.print("Enter City: ");
                String city = scanner.nextLine();

                pstmt.setInt(1, deptId);
                pstmt.setString(2, deptName);
                pstmt.setString(3, city);

                int rowsInserted = pstmt.executeUpdate();
                System.out.println(rowsInserted + " row(s) inserted successfully.\n");
            }

            // 2. Display Data using PreparedStatement
            String selectSQL = "SELECT * FROM Person";
            try (PreparedStatement pstmt = conn.prepareStatement(selectSQL);
                 ResultSet rs = pstmt.executeQuery()) {

                System.out.println("--- Table 'Person' Contents ---");
                System.out.printf("%-15s %-25s %-20s\n", "Department", "Department
Name", "City");
                System.out.println("----------------------------------------------------
-------------");
                while (rs.next()) {
                    System.out.printf("%-15d %-25s %-20s\n",
                        rs.getInt("Department"),
                        rs.getString("Department_Name"),
                        rs.getString("City"));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```
>_ Terminal: PreparedStatement Demo

$ java PreparedStatementDemo
[Connecting to MySQL server... Connected!]
Enter Department ID: 101
Enter Department Name: Human Resources
Enter City: Hyderabad
>> SUCCESS: Record inserted into Person table.

--- Fetching Content ---
+------------+----------------------+-----------+
| Department | Department Name      | City      |
+------------+----------------------+-----------+
| 101        | Human Resources      | Hyderabad |
+------------+----------------------+-----------+
```

3. Task 2: Delete Department using CallableStatement

Objective: Connect to the database and remove a department record from the Person table by invoking a stored procedure using a CallableStatement interface.

SQL Stored Procedure Definition:

```
DELIMITER //
CREATE PROCEDURE DeleteDepartment(IN deptId INT)
BEGIN
    DELETE FROM Person WHERE Department = deptId;
END //
DELIMITER ;
```

Java Source Code (DeleteDeptProcedure.java):

```
import java.sql.*;
import java.util.Scanner;

public class DeleteDeptProcedure {
    private static final String URL = "jdbc:mysql://localhost:3306/lab_db";
    private static final String USER = "root";
    private static final String PWD = "password123";
```

```
    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, USER, PWD);
             Scanner scanner = new Scanner(System.in)) {

            System.out.print("Enter Department ID to delete: ");
            int deptId = scanner.nextInt();

            // Preparing the call to the stored procedure
            String callSQL = "{call DeleteDepartment(?)}";
            try (CallableStatement cstmt = conn.prepareCall(callSQL)) {
                cstmt.setInt(1, deptId);

                int rowsDeleted = cstmt.executeUpdate();
                System.out.println("Stored procedure executed successfully.");
                System.out.println("Records matching Department ID " + deptId + " have
been removed.");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```
>_ Terminal: CallableStatement Demo

$ java CallableStatementDemo
[Connecting to DB server... Connected!]

Enter Department ID to delete: 101
>> Calling stored procedure: {call DeleteDepartment(?)}
>> SUCCESS: Stored Procedure completed cleanly.
Database confirmation: Row matching Department ID 101 removed.
Rows affected: 1
```

4. Task 3: Comprehensive CRUD Operations with Java

Objective: Illustrate full Create, Read, Update, and Delete (CRUD) actions on a database schema managing a Book catalog system.

SQL Table Creation Query:

```
CREATE TABLE Book (
    Serial_Number INT AUTO_INCREMENT PRIMARY KEY,
    Book_Number VARCHAR(50) UNIQUE,
    Book_Name VARCHAR(255),
    Price DECIMAL(10, 2)
);
```

Java Source Code (BookCRUDManager.java):

```
import java.sql.*;

public class BookCRUDManager {
    private static final String URL = "jdbc:mysql://localhost:3306/lab_db";
    private static final String USER = "root";
    private static final String PWD = "password123";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, USER, PWD)) {
            System.out.println("Connected to Database.\n");

            // 1. CREATE Operation
            System.out.println("--- Performing CREATE ---");
            String createSQL = "INSERT INTO Book (Book_Number, Book_Name, Price)
VALUES (?, ?, ?)";
            try (PreparedStatement pstmt = conn.prepareStatement(createSQL)) {
                pstmt.setString(1, "B-9981");
                pstmt.setString(2, "Introduction to Java Programming");
                pstmt.setDouble(3, 750.50);
                pstmt.executeUpdate();

                pstmt.setString(1, "B-4412");
                pstmt.setString(2, "Database System Concepts");
                pstmt.setDouble(3, 890.00);
                pstmt.executeUpdate();
                System.out.println("Books added successfully!\n");
            }

            // 2. READ Operation
            displayBooks(conn);

            // 3. UPDATE Operation
            System.out.println("\n--- Performing UPDATE ---");
            String updateSQL = "UPDATE Book SET Price = ? WHERE Book_Number = ?";
            try (PreparedStatement pstmt = conn.prepareStatement(updateSQL)) {
                pstmt.setDouble(1, 820.00); // New Price
```

```
                pstmt.setString(2, "B-9981"); // Target Book Number
                int updated = pstmt.executeUpdate();
                System.out.println(updated + " record updated successfully.\n");
            }

            // Read again to verify update
            displayBooks(conn);

            // 4. DELETE Operation
            System.out.println("\n--- Performing DELETE ---");
            String deleteSQL = "DELETE FROM Book WHERE Book_Number = ?";
            try (PreparedStatement pstmt = conn.prepareStatement(deleteSQL)) {
                pstmt.setString(1, "B-4412");
                int deleted = pstmt.executeUpdate();
                System.out.println(deleted + " record deleted successfully.\n");
            }

            // Read final time to verify deletion
            displayBooks(conn);

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    private static void displayBooks(Connection conn) throws SQLException {
        System.out.println("--- Performing READ (Current Book Inventory) ---");
        String selectSQL = "SELECT * FROM Book";
        try (PreparedStatement pstmt = conn.prepareStatement(selectSQL);
             ResultSet rs = pstmt.executeQuery()) {
            System.out.printf("%-10s %-15s %-35s %-10s\n", "Serial No", "Book No",
"Book Name", "Price");
            System.out.println("----------------------------------------------------------
------------------");
            while (rs.next()) {
                System.out.printf("%-10d %-15s %-35s %-10.2f\n",
                        rs.getInt("Serial_Number"),
                        rs.getString("Book_Number"),
                        rs.getString("Book_Name"),
                        rs.getDouble("Price"));
            }
        }
    }
}
```

```
>_ Terminal: CRUD Lifecycle Demo

$ java CRUDOperationsDemo
[CREATE]: Row inserted (Book: 5001, Java Fundamentals, Price: 650.0)
[READ]  : Current Database Grid Snapshot:
+-----------+----------+------------------+-------+
| Serial No | Book No  | Book Name        | Price |
+-----------+----------+------------------+-------+
| 1         | 5001     | Java Fundamentals | 650.0 |
+-----------+----------+------------------+-------+
[UPDATE]: Updating price for Book 5001 to 720.0... Success.
[DELETE]: Deleting row with Serial No 1... Complete.
CRUD Testing Complete.
```